Boolean logic as a harmonic filter

Dr. Greg Schiemer
Sydney Conservatorium of Music
Sydney, Australia.
gregs@conmusic.usyd.edu.au

Abstract

The author describes the motivation and method for generating harmonically related sets using boolean logic. The paper details an implementation in which boolean filters are used to generate scales and documents the process with musical examples.

1. Preamble

Boolean logic is a useful tool for culling surplus information. Its musical legacy dates back to some of the earliest composers involved in computer music including Xenakis, who developed the musical concept of set theory[1], and Martirano who built some of the first live performance music systems that used digital logic[2]. Boolean concepts have also been applied by composer-instrument-builders such as Burt[3], Vine[4] and the author[5] who built interactive hardware based on random logic integrated circuits.

The desire to use boolean instructions as a harmonic filter for MIDI note numbers was motivated by my involvement in the design of the MIDI Tool Box, purpose-built hardware for real-time musical applications using MIDI. Here minimal memory resources necessitated investigation of any means that might allow a composer to impose musical constraints on a system using a minimum of information. In the context of MIDI, where it takes 320 to transmit or receive a single byte, it is practical to apply such a harmonic filter in real time.

My own compositional use of boolean logic was based on the notion of getting more using less, and represents a paradigm that has now perhaps been overtaken since the arrival of the internet. Boolean logic is nowadays used in search engines as a way to extract what is sufficient from a superabundance of information.

2. The boolean concept of a harmonic filter

The set of MIDI note numbers [0,1,2,..,127] can also be expressed as sets of modulo-2 numbers; musically this representation allows the set to be thought of as two interleaved sets of whole-tone scales, a set of even numbers $[0,2,4,....,126]_{even}$ interleaved with a set of odd numbers $[1,3,5,....,127]_{odd}$. Figure 1 shows a two-octave section of a music keyboard with both interleaved whole-tone sets. MIDI note numbers in this figure are shown in decimal.

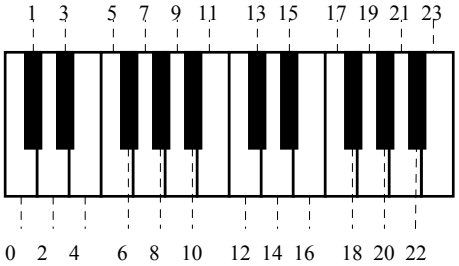


Fig. 1

Boolean operations are used to define members of either whole-tone scale set by

1 Xenakis, I [1992]: Formalized Music, Pendragon Press

2 Pellegrino, R. [1983]: Sal-Mar Construction (Martirano), pp. 95-98

3 Aardvarks Machine in Burt, W [1991]: Experimental Music in Australia using Live Electronics CMR, Vol. 6 Part 1, eds. P. Nelson & S. Montague, pp. 168-171

4 Vine, C. [1978]: Patent Little Marvel, New Music Magazine, Latrobe U., ed. W. Burt, pp. 6-7

5 Tupperware Gamelan in Schiemer, G. [2000]: Improvising Machines: Spectral Dance and Token Objects, Leonardo Music Journal, ed. N.Collins, Vol.9, pp.107-114

masking out or hiding those bits that do not belong to the set.
Because they allow modulo-2 calculations to be performed very efficiently, boolean operations are included as part of the instruction set of most microprocessors and digital signal processors.

In the harmonic filter, two types of boolean operation are used: logical AND and logical OR. Appendix A.1. illustrates both these as 1-bit binary operations; Appendix A.2 illustrates boolean AND as a 7-bit binary operation.

Logical AND is used to define the whole-tone set of even numbers using an even numbered mask 111 1110 [126] $_{even}$ while logical OR is used to define the whole-tone set of odd numbers using an odd numbered mask 000 0001 [1] $_{odd}$.

Other masks can be used to hide non-members of the various subsets of both whole-tone sets. This allows more sophisticated harmonic filters to be developed. For example, the even mask 111 1100 [124] $_{even}$ and the odd mask 000 0011 [3] $_{odd}$ will both produce steps of a major third, while the even mask 111 1000 [120] $_{even}$ and the odd mask 000 0111 [7] $_{odd}$ will both produce steps of a tritone.

The boolean concept of harmonic filter potentially allows any chord to be expressed as an intersection of the even and odd sets of whole-tone scales or their sub-sets. For example, a major triad can be expressed as the intersection of [0, 4]$_{even}$ and [7]$_{odd}$ for the lower octave and [12, 16]$_{even}$ and [19]$_{odd}$ for the upper octave as shown in Fig. 2, while the minor triad can be expressed as the intersection of [0]$_{even}$ and [3, 7]$_{odd}$ for the lower octave and [12]$_{even}$ and [15, 19]$_{odd}$ for the upper octave as shown in Fig.3.

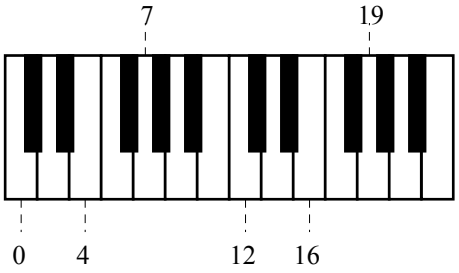


Fig. 2

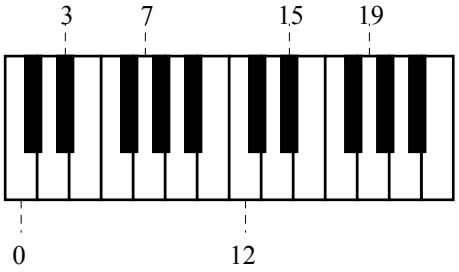


Fig. 3

The technique of defining harmonic filters using boolean operations, though computationally efficient, requires a simple modification to work with the set of modulo-12 numbers, called pitch classes, which has been the foundation of western harmony for the past few centuries. Integer division allows a set of pitch classes [0,1,2,...,11] to be extracted from a set of MIDI note numbers; with a divisor of 12, the quotient is the pitch class, the dividend is the octave. Once the harmonic filter has been applied, the filtered pitch class is then recombined with (ie. multiplied by) the octave to produce a filtered version of the original MIDI note number. Only one pitch class extraction is required for each MIDI note number received or transmitted. This allows harmonic filtering to be applied in conventional 12-note-per-octave harmony, and makes it practical for real-time musical applications.

However before further examining the feasibility of conventional chords within the 12-note- per-octave system, it seemed necessary to explore the musical possibilities of modulo-2 numbers first. Pitch class extraction would limit such exploration to chords formed from octave-limited scales ie. scales confined to a range

of one octave. There is plenty we already know about harmony produced from octave-limited scales. There is much more to learn about harmony generated using boolean operations.

3. Boolean harmonic filter algorithm

The following algorithm was developed. For each note :

- MIDI note number is duplicated once for odd mask & once for even mask
- odd mask is ORed with MIDI note number to form odd partial result
- even mask is ANDed with MIDI note number to form even partial result
- both partial results are interrogated to detect membership of either set

These steps are repeated for every MIDI note number until two octaves have been played.
Appendix B shows the relevant extract of a documented listing of the algorithm as it was implemented in HC11 assembler code for the MIDI Tool Box (MTB).

3.1. Method

The above implementation of the boolean filter algorithm was cross-assembled and down-loaded into the MTB. The program was run several times. Each time the program was run, new values for the even mask and the odd mask were manually entered in the program prior to restarting.

The musical behaviour of boolean operations was observed by listening to the effect of various logical AND and logical OR masks on a series of MIDI note numbers played in ascending order. Without using pitch-class and octave separation, the boolean filter algorithm described above was found to generate interesting scales. Several of these are octave-limited, most are non-octave-limited.

Data samples shown in Appendix C were recorded as a screen dump on the MTB each time the program was run. These form a representative selection of samples of the musical scales that can be synthesised using this technique. They are shown in hexadecimal (ie. modulo-16) notation. Some of these scales can be produced using more than one set of masks. For instance, example C.1, C.2, C.3 & C.4 are the same scale produced using different masks; likewise, for example C.5, C.6 & C.8

Scale steps are expressed chromatically, where a single step represents a minor 2nd (m2), two steps, a major 2nd (M2), three steps, a minor 3rd (m3) and so on. The first few examples are chromatic ie. consisting of all minor 2nds. All examples shown have a span of two octaves, spanning note numbers from two octaves below middle C (36 or, in hexadecimal, $24) to middle C (60 or $3C). This allows sufficient steps to determine whether or not the scale is octave-limited.

Because the MTB screen dump is in hexadecimal a few of the more interesting examples are also presented here using staff notation. One is octave-limited the others non-octave-limited.

Figure 4 (corresponding to example C.5) is like the octotonic scale except that it contains two semitone steps for each whole-tone step. Like the octotonic scale, it is octave-limited. Figure 5 (example C.10) is a non-octave-limited scale with three whole tone steps for each pair of semitone steps. Figure 6 (example C.19) is another non-octave-limited scale in which a whole tone step is separated from a minor 3rd by one semi-tone step in one direction and two semi-tone steps in the other direction.

Example No. 5. Generator: M2, m2, m2

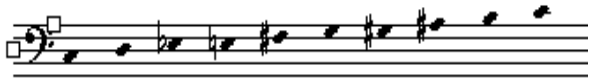


Fig. 4

Example No. 10. Generator: M2, m2, m2, M2, M2



Fig.5

Example No. 19. Generator: m2, M2, m2, m2, m3



Fig.6

4. Future development

4.1. Applications

One of the proposed changes to the harmonic filter algorithm included separation of MIDI note number into pitch class and octave as outlined in 2. This would provide a way to define groups of notes using just two bytes making it possible to identify chord patterns quickly in live performance. By comparing a note played with a filtered version of the same note, one can establish whether or not the filtered and unfiltered versions are identical; chordal identity of a note can be established in less time than it takes to transmit or receive the next byte of MIDI.

With or without the inclusion of pitch class, the algorithm would also be useful in algorithmic compositions to filter random notes in a harmonically ordered way. Subtle changes to the harmonic field can be made by applying ramp, and triangular functions etc. to manipulate one or other, or both masks in realtime. Examples of this can already be demonstrated.

4.2. Platforms

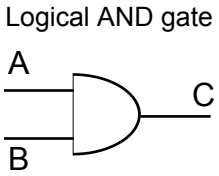
There is no reason for this algorithm to be limited to the MIDI Tool Box even though this has been an ideal platform for my own development and compositional work with MIDI. Others hopefully will find ways of implementing this algorithm by creating Max objects or writing grammars using the Bol Processor. In such computing environments where MIDI and audio signal processing meet, harmonic filters have applications that go beyond the availability of 12 notes in every octave.

5. Conclusion

Regardless of computer platform or software application, boolean operations provide a new way to interpret or manipulate harmonic information in real time. For me, any further musical experimentation with these operations as harmonic filters will not be confined to the 12-tone equal tempered system.

Appendix A.1. 1-bit

Boolean functions use binary logic.
Figure 7a shows a logic circuit called a logical AND-gate, where both inputs must be true (ie. = 1) for the output to be true. Figure 7b shows the truth table for an AND-gate. Figure 8a shows a logic circuit called a logical OR-gate, where, if either input is true, then the output must be true. Figure 8b shows the truth table for an OR-gate.

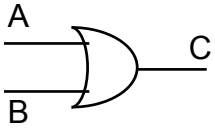


[a] AND circuit

inputs		output
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

[b] AND truth table

Fig. 7



[a] OR circuit

inputs		output
A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

[b] OR truth table

Fig. 8

Appendix A.2. 7-bit

Boolean operations are simply a collection of binary gates that operate in parallel. Figure 9 shows how an odd MIDI note number 7 (binary 000 0111 in input register A) is transformed using boolean AND operation with a 7-bit binary mask of 111 1110 (in input register B) to produce an even MIDI note number 6 (binary 000 0110 in output register C). As long as the value in register B remains 111 1110, then no matter what 7-bit binary value is contained in register A, the output register will always contain an even number, ie. the least significant bit of register C will always equal zero.

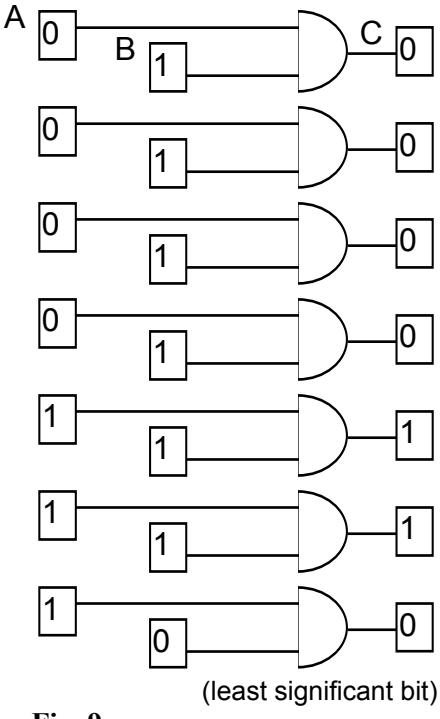


Fig. 9

The same principle of 7 binary gates that operate in parallel applies for odd numbers using the boolean OR operation.

```
MIDI_COMMAND    EQU     $90   (144 : MIDI Status Byte)
STARTING_NOTE   EQU     $24   (36 :  MIDI note number)
AUDIBLE_LEVEL   EQU     $7F   (127 : MIDI key velocity)
NOTE_LENGTH     EQU     $18   (note length approx. 0.25 sec)
NOTE_FILTER1    EQU     $7F   AND_MASK
NOTE_FILTER2    EQU     $0    OR_MASK

        ORG     $3000
STATUS_BYTE     RMB     $1    (1-byte variable)
NOTE_NUMBER     RMB     $1    (1-byte variable)
KEY_VELOCITY    RMB     $1    (1-byte variable)
DURATION        RMB     $1    (1-byte variable)
EVEN_MASK       RMB     $1    (1-byte variable)
ODD_MASK        RMB     $1    (1-byte variable)
*
*-----------------------------------------
        ORG     $2000
        JSR     POLLM1          Select Polling [ROM Tool]
        JSR     INIT_REGS       Setup program controls
*                               Start of Main Program
MAIN    JSR     SET_LENGTH      (Re-)set Note length
        JSR     KON             Turn MIDI Note ON
        JSR     HOLD            Hold it on
        JSR     KOFF            Turn MIDI Note OFF
        JSR     NEXT_NOTE       Generate next note
        BRA     MAIN            Repeat the main loop

*------  User Subroutine 6
*       Generate next Note
NEXT_NOTE
        LDAA    NOTE_NUMBER     Now try next note
        INCA                    and keep this one
        ANDA    #$7F            within the limits
        STAA    NOTE_NUMBER     of Note Numbers
        JSR     PITCH_FILTER    and filter it
        RTS

*------  User Subroutine 7
*       Note-numbers pass through filter only when these are members
*       of either set of pitches defined by the two mask patterns

PITCH_FILTER
        LDAA    NOTE_NUMBER     Get Note-number
        TAB                     in AccA & AccB
        ANDA    EVEN_MASK       Check for set of
        CBA                     EVEN Whole-tones
        BEQ     NOTE_PASS
        TBA                     Get Note-number
        ORAA    ODD_MASK        Check for set of
        CBA                     ODD Whole-tones
        BEQ     NOTE_PASS
        INC     NOTE_NUMBER     Try again with
        BRA     PITCH_FILTER    the next one

        NOTE_PASS
*                               Next one worked!
        ANDA     #$7F           Keep within limits
        STAA    NOTE_NUMBER     of Note Numbers
        RTS
```

Appendix C Musical examples

C.1 AND = $7F; OR = $00

Chromatic

```
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 29 2A 2B  2C 2D 2E 2F  30 31 32 33  $%&'()*+,-./0123
7010  34 35 36 37  38 39 3A 3B  3C
```

C.2 AND = $7F; OR = $01

Chromatic

```
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 29 2A 2B  2C 2D 2E 2F  30 31 32 33  $%&'()*+,-./0123
7010  34 35 36 37  38 39 3A 3B  3C
```

C.3 AND = $7E; OR = $00

Chromatic

```
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 29 2A 2B  2C 2D 2E 2F  30 31 32 33
7010  34 35 36 37  38 39 3A 3B  3C
```

C.4 AND = $7E; OR = $01

Chromatic

```
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 29 2A 2B  2C 2D 2E 2F  30 31 32 33  $%&'()*+,-./0123
7010  34 35 36 37  38 39 3A 3B  3C
```

C.5 AND = $7E; OR = $02

M2 m2 m2 M2 m2 m2 M2 m2 m2 M2 m2 m2 M2 m2 m2 M2
m2 m2

```
0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 27 28  2A 2B 2C 2E  2F 30 32 33  34 36 37 38  $&'(*+,./0234678
7010  3A 3B 3C
```

C.6 AND = $7E; OR = $03

M2 m2 m2 M2 m2 m2 M2 m2 m2 M2 m2 m2 M2 m2 m2 M2
m2 m2

```
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 27 28  2A 2B 2C 2E  2F 30 32 33  34 36 37 38  $&'(*+,./0234678
7010  3A 3B 3C
```

C.7 AND = $7E; OR = $04

m2 m2 m2 m2 M2 M2 m2 m2 m2 m2 M2 M2 m2 m2 m2 m2
M2 M2

```
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 2A 2C 2D  2E 2F 30 32  34 35 36 37  $%&'(*,-./024567
7010  38 3A 3C
```

C.8 AND = $7E; OR = $05

M2 m2 m2 M2 m2 m2 M2 m2 m2 M2 m2 m2 M2 m2 m2 M2
m2 m2

```
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 27 28  2A 2B 2C 2E  2F 30 32 33  34 36 37 38  $&'(*+,./0234678
7010  3A 3B 3C
```

C.9 AND = $7E; OR = $06

```
          M2 m2 m2 M2  M2 M2 m2 m2  M2 M2 M2 m2  m2 M2 M2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 27 28  2A 2C 2E 2F  30 32 34 36  37 38 3A 3C  $&'(*,./024678:<
7010
```

C.10 AND = $7E; OR = $07

```
          M2 m2 m2 M2  M2 M2 m2 m2  M2 M2 M2 m2  m2 M2 M2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 27 28  2A 2C 2E 2F  30 32 34 36  37 38 3A 3C  $&'(*,./024678:<
7010
```

C.11 AND = $7E; OR = $08

```
          M2 M2 m2 m2  m2 m2 m2 m2  m2 m2 M2 M2  M2 M2 m2 m2
          m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 28 29  2A 2B 2C 2D  2E 2F 30 32  34 36 38 39  $&()*+,-./024689
7010  3A 3B 3C
```

C.12 AND = $7E; OR = $09

```
          M2 M2 m2 m2  m2 m2 m2 m2  m2 m2 M2 M2  M2 M2 m2 m2
          m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 28 29  2A 2B 2C 2D  2E 2F 30 32  34 36 38 39  $&()*+,-./024689
7010  3A 3B 3C
```

C.13 AND = $7E; OR = $0A

```
          M2 M2 M2 m2  m2 M2 m2 m2  M2 M2 M2 M2  M2 m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 28 2A  2B 2C 2E 2F  30 32 34 36  38 3A 3B 3C  $&(*+,./02468:;<
7010
```

C.14 AND = $7E; OR = $0B

```
          M2 M2 M2 m2  m2 M2 m2 m2  M2 M2 M2 M2  M2 m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 26 28 2A  2B 2C 2E 2F  30 32 34 36  38 3A 3B 3C  $&(*+,./02468:;<
7010
```

C.15 AND = $7D; OR = $01

```
          m2 M2 m2 m2  M2 m2 m2 M2  m2 m2 M2 m2  m2 M2 m2 m2
          M2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 27 28  29 2B 2C 2D  2F 30 31 33  34 35 37 38  $%'()+,-/0134578
7010  39 3B 3C
```

C.16 AND = $7D; OR = $02

```
          Chromatic
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 29 2A 2B  2C 2D 2E 2F  30 31 32 33  $%&'()*+,-./0123
7010  34 35 36 37  38 39 3A 3B  3C
```

C.17 AND = $7D; OR = $03

```
          m2 M2 m2 m2  M2 m2 m2 M2  m2 m2 M2 m2  m2 M2 m2 m2
          M2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 27 28  29 2B 2C 2D  2F 30 31 33  34 35 37 38  $%'()+,-/0134578
7010  39 3B 3C
```

C.18 AND = $7D; OR = $04

```
          m2 m2 m2 m2  m2 m3 m2 m2  m2 m2 m2 m3  m2 m2 m2 m2
          m2 m3
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 29 2C 2D  2E 2F 30 31  34 35 36 37  $%&'(),-./014567
7010  38 39 3C
```

C.19 AND = $7D; OR = $05

```
          m2 M2 m2 m2  m3 m2 M2 m2  m2 m3 m2 M2  m2 m2 m3
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 27 28  29 2C 2D 2F  30 31 34 35  37 38 39 3C  $%'(),-/0145789<
7010
```

C.20 AND = $7D; OR = $06

```
          m2 m2 m2 m2  m2 m3 m2 m2  m2 M2 m2 m3  m2 m2 m2 m2
          m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 26 27  28 29 2C 2D  2E 2F 30 31  34 35 36 37  $%&'(),-./014567
7010  38 39 3C
```

C.21 AND = $7D; OR = $07

```
          m2 M2 m2 m2  m3 m2 M2 m2  m2 m3 m2 M2  m2 m2 m3
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 27 28  29 2C 2D 2F  30 31 34 35  37 38 39 3C  $%'(),-/0145789<
7010
```

C.22 AND = $7D; OR = $08

```
          m2 m3 m2 m2  m2 m2 m2 m2  m2 m2 m2 m3  m2 m3 m2 m2
          m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 28 29  2A 2B 2C 2D  2E 2F 30 31  34 35 38 39  $%()*+,-./014589
7010  3A 3B 3C
```

C.23 AND = $7D; OR = $09

```
          m2 m3 m2 M2  m2 m2 M2 m2  m2 m3 m2 m3  m2 M2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 28 29  2B 2C 2D 2F  30 31 34 35  38 39 3B 3C  $%()+,-/014589;<
7010
```

C.24 AND = $7D; OR = $0A

```
          m2 m3 m2 m2  m2 m2 m2 m2  m2 m2 m2 m3  m2 m3 m2 m2
          m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 28 29  2A 2B 2C 2D  2E 2F 30 31  34 35 38 39  $%()*+,-./014589
7010  3A 3B 3C
```

C.25 AND = $7D; OR = $0B

```
          m2 m3 m2 M2  m2 m2 M2 m2  m2 m3 m2 m3  m2 M2 m2
          m2 m2
      0  1  2  3   4  5  6  7   8  9  A  B   C  D  E  F        ascii
7000  24 25 28 29  2B 2C 2D 2F  30 31 34 35  38 39 3B 3C  $%()+,-/014589;<
7010
```